

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Departamento de Ciencias e Ingeniería de la Computación

Fundamentos de Programación

Practica 9 Memoria dinámica

Para resolver los siguientes problemas, sus programas en lenguaje C, usted debe resolver con el uso de apuntadores y de memoria dinámica (No olvide emplear apuntador a memoria dinámica), así como todas sus funciones no devuelven nada y el paso de argumentos se realiza por referencia. Realice sus diagramas de flujo antes de programar, presente su diagrama antes del código y de la ejecución.

1. Empleando aritmética de apuntadores en todas las funciones, desarrolle un vector de 50 números enteros que sea leído desde una función desde el teclado o sean creados de forma aleatoria, incluya una función que muestre el vector y cree otra función que le pida al usuario un número y que lo busque dentro del vector y si lo encuentra que devuelva la posición donde lo encontró (si existen más elementos buscados en el vector deberá mostrarlos), en caso contrario que devuelva un número negativo y de acuerdo a esto le diga al usuario lo ocurrido. También cree otras dos funciones, una para ordenar el vector de forma ascendente y otra para ordenarlos de forma descendente. Emplee un menú para realizar las tareas.

2. Escriba un programa que lea una frase de varias líneas de texto (1000 caracteres como mínimo) y genere una lista de todas las letras que aparecen en el texto junto con el número de veces que cada letra ocurre en la línea. Termina la línea con un punto que sirva como valor centinela. Las letras deben enumerarse en el siguiente orden: la letra más frecuente, la siguiente letra más frecuente, y así sucesivamente. Usar dos matrices (dinámicas), una para contener los números enteros y otra para contener las letras, puede asumir que la entrada utiliza todas las letras minúsculas. Por ejemplo, la entrada (este ejemplo no cumple lo de frase, solo es demostrativo)

do be do bo

debe producir una salida similar a la siguiente:

Letra Numero de ocurrencias

o	3
d	2
b	2
e	1

Su programa necesitará ordenar los arreglos de acuerdo a los valores en una matriz de enteros (un vector de apuntador asignado con memoria dinámica). Esto requerirá que modifique el tipo de función. No puede usar ordenar para resolver este problema sin cambiar la función. Si se trata de una tarea de clase, pregúntele a su instructor si la entrada/salida se debe hacer con el teclado y la pantalla o si se debe terminar con los archivos. Si se va a hacer con archivos, pídale a su instructor instrucciones sobre nombres de archivo.

3. Del siguiente código, aplique el teorema de modularidad y cree los métodos necesarios para que el programa trabaje como trabaja actualmente, recuerde que en el main() solo debe haber el llamado a una función que empiece a ejecutar el programa, emplee apuntadores, paso por referencia y memoria dinámica.

```
1  #include <stdio.h>
2
3  #define MAX 50
4  #define N 3
5  #define M 4
6
7  int main(int argc, char *argv[]) {
8      int i=0, j=0, matriz[MAX][MAX], auxiliar, mayor;
9      if(N>=M) {
10         mayor = N;
11     }
12     else {
13         /* es necesario para poder decir cuantas veces se hara un
14         intercambio del elemento (i,j) al elemento (j,i) */
15         mayor=M;
16     }
17
18     for(i=0; i<N; i++) {
19         for(j=0; j<M; j++) {
20             printf("\nIngrese el elemento [%d],[%d] : ",i,j);
21             scanf("%d",&matriz[i][j]);
22         } /* Aquí se leyeron todos los elementos de la matriz */
23     }
24     printf("\n\nSu Matriz original es : ");
```

```
25     for(i=0; i<N; i++) {
26         printf("\n");
27         for(j=0; j<M; j++) {
28             printf("%d\t",matriz[i][j]);
29         }
30     } /* Aquí simplemente se imprimieron los elementos de la matriz */
31     for(i=0; i<mayor; i++) {
32         for(j=i+1; j<mayor; j++) {
33             auxiliar = matriz[j][i];
34             matriz[j][i] = matriz[i][j];
35             matriz[i][j] = auxiliar;
36         } /* Aquí la matriz se convirtio en su traspuesta */
37     }
38     printf("\n\nSu Matriz Traspuesta es : ");
39     for(i=0; i<M; i++) {
40         printf("\n");
41         for(j=0; j<N; j++) {
42             printf("%d\t",matriz[i][j]);
43         }
44     }
45     return 0;
46 }
```

4. Crear un programa que permita mantener una lista con los datos de los obreros de una empresa, para ello suponga que el máximo de obreros es de 40, emplee memoria dinámica para almacenar a los obreros. Además, agregue los datos correspondientes a, número de horas extra que hace en el mes, el valor por hora extra que se le paga a cada obrero y el total de sobresueldo (horas extra por valor hora). Realice las siguientes operaciones: Ingresar el total de datos para los 40 obreros, luego imprima una lista resumida de los datos (sólo nombre, horas extra, valor por hora y total).

Emplee la siguiente estructura

```
/* definicion de estructura */
struct Obrero {
    char nombre[60]; /*incluido apellidos*/
    char direccion[60]; /*Calle + numero + colonia*/
    int horasExtra; /*Horas Extras*/
    int valorPorHora; /*Valor por Hora*/
    float total; /* horasExtra + valorPorHora */
    float salario; /*Salario del obrero*/
};
```

Cree un menú para realizar las operaciones de agregar obrero, ver a los obreros, modificar la información de un obrero, agregar horas extra, recuerde sumar el pago de horas extra al salario del obrero mediante una función.